Fridgo

(Smart Fridge & Chief)

Team Members:

Ahmed Ayman Elbamby

Ali Osama Ali

Salma Mohammed Hassan

Mohamed Emad Ahmed

Moahmed Tareq Mahmoud

Mohamed Reda Hashish

Supervised By:

Eng.Alaa Samir

Contents

Table of Figures

CHAPTER 1

Introduction

1.1 Project Overview

Fridgo is a graduation project centered on an artificial intelligence system that identifies food ingredients from a photograph of a refrigerator and recommends recipes that can be prepared from the identified ingredients. The system also provides a conversational assistant that answers questions about a selected recipe and can rewrite a recipe according to a user's request, such as adapting it to a dietary restriction or a missing ingredient.

The project is composed of an AI module, which performs ingredient detection, recipe ranking, and conversational recipe assistance, and a web-based frontend, which exists to demonstrate the AI module's capabilities through an interactive interface.

1.2 Problem Statement

Deciding what to cook based on the ingredients currently available at home is a recurring and often time-consuming task. Ingredients kept in a refrigerator are frequently forgotten, misidentified, or left unused until they spoil, while manually searching for recipes that match a specific and variable set of available ingredients requires effort that many users are unwilling to spend. This project addresses that problem by automating the identification of available ingredients from a single photograph and by automatically ranking candidate recipes according to how well they match the ingredients that were detected or manually specified by the user. By automating this process, the system helps reduce food waste, saves time, and simplifies everyday meal planning.

1.3 Objectives

The project sets out to achieve the following objectives, each of which corresponds to a capability implemented in the system:

- Automatically detect food ingredients present in a fridge photograph using a computer-vision pipeline, without requiring the user to manually enumerate the contents of the fridge.
- Allow the user to review and correct the automatically detected ingredient list by adding items that were missed or removing false positives before requesting recipes.
- Rank and recommend recipes from a large recipe dataset according to how well each recipe's ingredient list is covered by the ingredients the user has available.
- Provide a conversational assistant capable of answering questions about a selected recipe and of modifying that recipe's ingredients and steps in response to natural-language requests.
- Expose the AI module's functionality through a web-based interface that demonstrates the complete workflow, from image upload to recipe recommendation to conversational interaction.

1.4 Scope

This project focuses on developing an AI-powered system that detects food ingredients from refrigerator images, recommends suitable recipes, and provides a conversational assistant for recipe-related interactions. The system is implemented as a functional prototype intended to demonstrate the feasibility of the proposed approach. Features such as user authentication, persistent data storage, and production deployment are outside the scope of this project.

1.5 Technologies Used

The project combines a lightweight web frontend with a Python-based AI backend. The technologies used are listed below; their role within the system architecture is discussed in later chapters.

- Frontend: HTML5, CSS3, and vanilla JavaScript, with Google Fonts and the Material Symbols icon set.
- Backend framework: FastAPI, served with Uvicorn, and exposed publicly through an ngrok tunnel.
- Computer vision: PyTorch and Hugging Face Transformers, using a Grounding DINO open-vocabulary object detector and a CLIP model for detection verification, together with OpenCV and the supervision library for image processing and annotation, and ensemble-boxes for Weighted Boxes Fusion.
- External detection service: a custom-trained ingredient detection model hosted on Roboflow, called as a complementary detection source.
- Recipe search: sentence-transformers with a FAISS similarity index for semantic search, and rank_bm25 for lexical search, combined over a recipe dataset obtained through kagglehub, with pandas and NumPy used for data handling.
- Conversational assistant: the Groq API, using the llama 3.3 70b versatile language model.

1.6 Project Structure Overview

The project's source files are organized into a single application directory containing five subfolders, reflecting the separation between the AI module and the demonstration frontend:

- App/AI/ — contains the notebook implementing the entire AI backend: ingredient detection, recipe ranking, the conversational assistant, and the FastAPI server that exposes them.
- App/pages/ — contains the HTML pages of the demonstration frontend: the landing page, the main application dashboard (the functional page where images are uploaded and recipes are requested), and an about-us page.

- App/js/ — contains the JavaScript file that implements all client-side logic and communicates with the AI backend's API.
- App/css/ — contains the stylesheet shared by all frontend pages.
- App/imgs/ — contains static image assets used by the frontend, such as the project logo and team photographs.

The following chapters provide a detailed description of the system architecture, the data flow between the frontend and the AI backend, and the algorithms implemented in the AI module.

CHAPTER 2
System Analysis and Design

2.1 Existing Problem

Chapter 1 introduced the general motivation for Fridgo (the project's internal / notebook name, shown in the source as fridgo_backend.ipynb; the demonstration frontend brands the same system as "Fridgo"). This section analyses the existing, manual process the system replaces, as a basis for the design decisions in the rest of this chapter.

Without a system such as this one, a person deciding what to cook must perform three steps entirely manually: (1) physically inspect the fridge and mentally enumerate its contents, (2) recall or search for a recipe using that mental list, matching ingredients by memory rather than by any verified check, and (3) adapt the recipe by hand if an ingredient is missing or unsuitable, without any assistance. None of these three steps is automated, verified, or assisted by any tool in the baseline (no-system) case. Analysed as a system, the manual process has no defined "input capture" step (ingredients are not recorded anywhere), no ranking function (recipe suitability is judged subjectively), and no adaptation mechanism (changes to a recipe must be reasoned out unaided).

2.2 Proposed Solution

Fridgo replaces each of the three manual steps identified above with corresponding automated components:

1. In place of manual inspection, an image-based ingredient detection pipeline (Section 2.8) accepts a photograph and returns a structured ingredient list, combining an open-vocabulary vision-language detector (Grounding DINO) with a separately hosted, custom-trained detector (Roboflow), fused and verified across multiple stages before being returned to the user.
2. In place of unaided recall, a hybrid recipe retrieval engine (BM25 lexical search combined with FAISS semantic search) ranks a 15,000-recipe dataset against the ingredient list and reports an explicit, computed match/coverage percentage per recipe, rather than leaving suitability to be judged subjectively.
3. In place of unaided adaptation, a recipe-scoped chatbot (Groq-hosted Llama 3.3 70B) accepts natural-language requests against a specific, selected recipe — such as removing an ingredient or adjusting for a dietary need — and returns a complete, rewritten recipe rather than only a text explanation.

The frontend (three static HTML pages, plain CSS, and vanilla JavaScript) exists to drive this backend end-to-end for demonstration purposes: it lets the user supply a photo, correct the detected list, trigger a recipe search, and converse with the chatbot, without introducing any AI logic of its own.

2.3 Functional Requirements

The functional requirements listed below are based on the implemented system functionalities and describe the core services provided by the application.

- **FR-1:** The system shall accept an uploaded image file (JPEG, PNG, or WEBP, up to 15 MB) and return a list of detected ingredient labels.
- **FR-2:** The system shall reject unsupported file types and oversized files with an appropriate error message before processing the image.
- **FR-3:** The system shall analyze the uploaded image using multiple ingredient detection techniques to improve the accuracy of the detected results.
- **FR-4:** The system shall allow the user to manually add ingredient names to the detected ingredient list.
- **FR-5:** The system shall allow the user to remove detected or manually added ingredients before searching for recipes.
- **FR-6:** The system shall accept the final ingredient list and return up to three ranked recipe recommendations, including the recipe title, ingredients, preparation steps, matched ingredients, and a match percentage.
- **FR-7:** The system shall allow the user to select one of the recommended recipes as the active context for the conversational assistant.
- **FR-8:** The system shall accept natural-language queries related to the selected recipe and return appropriate conversational responses.
- **FR-9:** When requested by the user, the system shall generate a modified version of the selected recipe based on the user's requirements, such as dietary preferences or ingredient substitutions.
- **FR-10:** The system shall provide appropriate feedback and error messages whenever a requested operation cannot be completed successfully.

2.4 Non-Functional Requirements

The following non-functional requirements describe the quality attributes considered in the design and implementation of the proposed system.

- **Performance:** The system is designed to provide timely responses for ingredient detection, recipe retrieval, and conversational interactions, ensuring a smooth user experience under normal operating conditions.
- **Usability:** The web interface is designed to be simple and intuitive, allowing users to upload images, edit detected ingredients, search for recipes, and interact with the chatbot without requiring technical knowledge.
- **Reliability:** The ingredient detection pipeline incorporates multiple detection and verification stages to improve the robustness of the results and reduce the likelihood of incorrect detections.
- **Interoperability:** The backend exposes RESTful APIs that enable seamless communication with the web frontend and integration with external AI services used for ingredient detection and conversational assistance.

- **Maintainability:** The backend is organized into separate functional modules for ingredient detection, recipe retrieval, and chatbot services, making the system easier to maintain and extend.
- **Scalability:** The current implementation is designed as a functional prototype. Future versions can be extended with persistent storage, user authentication, and cloud deployment to support larger numbers of users.
- **Security:** Basic communication between the frontend and backend is supported through API-based interactions. Advanced security mechanisms, including authentication, authorization, and secure credential management, are considered outside the scope of the current prototype and are recommended for future development.

2.5 Overall Architecture

Fridgo follows a simple layered architecture consisting of a presentation layer, an API/service layer, and an AI processing layer that provides the three core functionalities: ingredient detection, recipe retrieval, and conversational assistance. Figure 2.1 illustrates the overall system architecture and the interaction between these components.

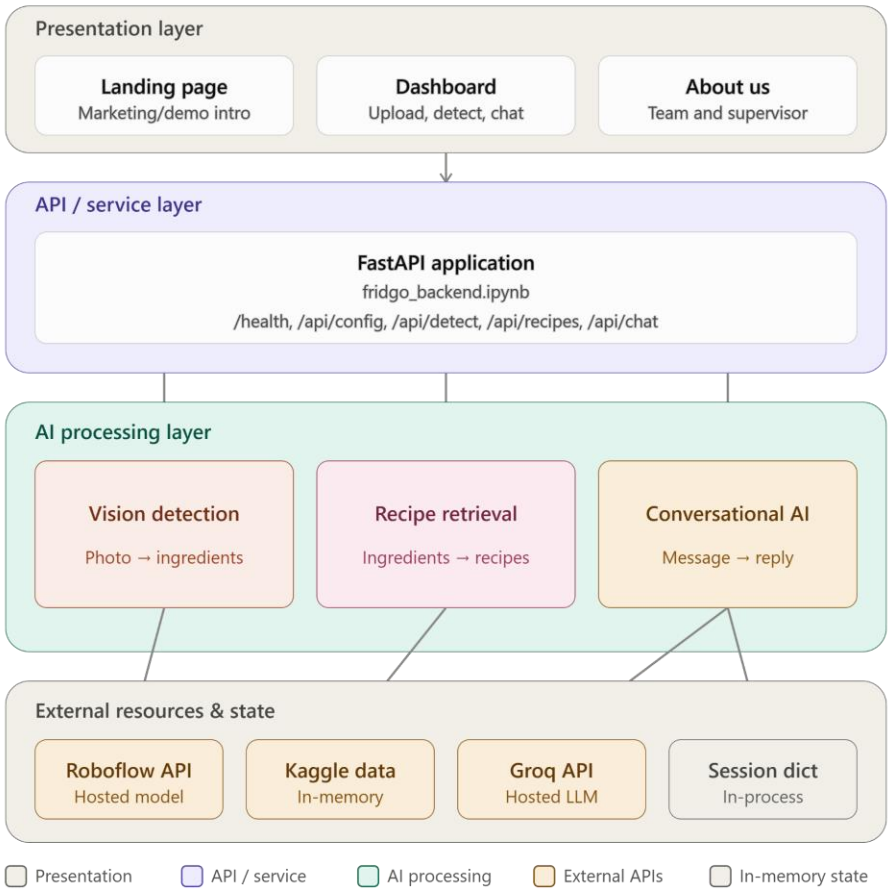

Figure 1:Overall system architecture, showing the presentation, API, AI-processing, and data/external-services layers.

2.6 Folder Structure

Chapter 1 presented the project's physical folder tree. This section instead maps each folder to the architectural layer it implements, which is the relevant view for system design.

Folder / File	Architectural Layer	Responsibility
pages/*.html	Presentation	Three static pages: landing page, functional dashboard, about page
css/style.css	Presentation	Shared visual styling for all pages
js/script.js	Presentation	Client-side logic: file upload, DOM rendering, calls to all four backend endpoints
AI/fridgo_backend.ipynb — API section	API / Service	FastAPI app definition, CORS, the five route handlers
AI/fridgo_backend.ipynb — detection section	AI Processing	Grounding DINO, Roboflow call, WBF fusion, CLIP verification
AI/fridgo_backend.ipynb — recipe section	AI Processing	BM25 + FAISS hybrid retrieval, recipe coverage scoring
AI/fridgo_backend.ipynb — chat section	AI Processing	Groq API integration, chat system prompt, session handling
imgs/	Presentation (static assets)	Logo and team photography; no functional role

Table 1: Folder-to-architecture mapping (system-design view of the project structure).

2.7 Data Flow

Figure 2.2 illustrates how data flows between the user, the backend components, the system data stores, and the external AI services (Roboflow and Groq) during the ingredient detection, recipe recommendation, and conversational assistance processes.

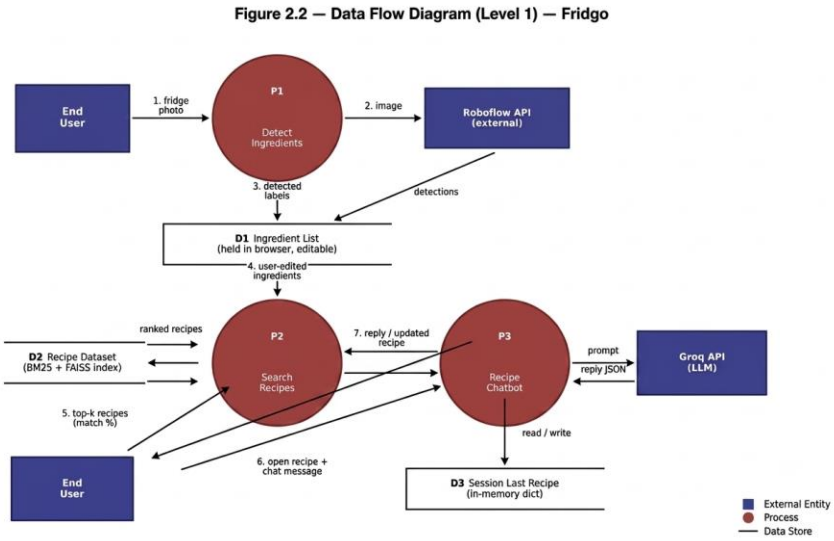

Figure 2:Level 1 Data Flow Diagram for Fridgo's detect → search → chat sequence.

Two important characteristics of the data flow should be noted. First, the ingredient list is maintained on the client side throughout the user's session and is not stored persistently on the server. Second, the chatbot context is associated with the current user session, allowing recipe-related conversations to remain consistent during the active session without requiring user authentication or permanent data storage.

2.8 AI Workflow Overview

This section provides a high-level overview of the AI workflows implemented in the system, including ingredient detection, recipe retrieval, and conversational recipe assistance. Figure 2.3 illustrates how these components interact to provide an integrated AI-powered workflow. Detailed descriptions of the underlying models, algorithms, and evaluation methods are presented in later chapters.

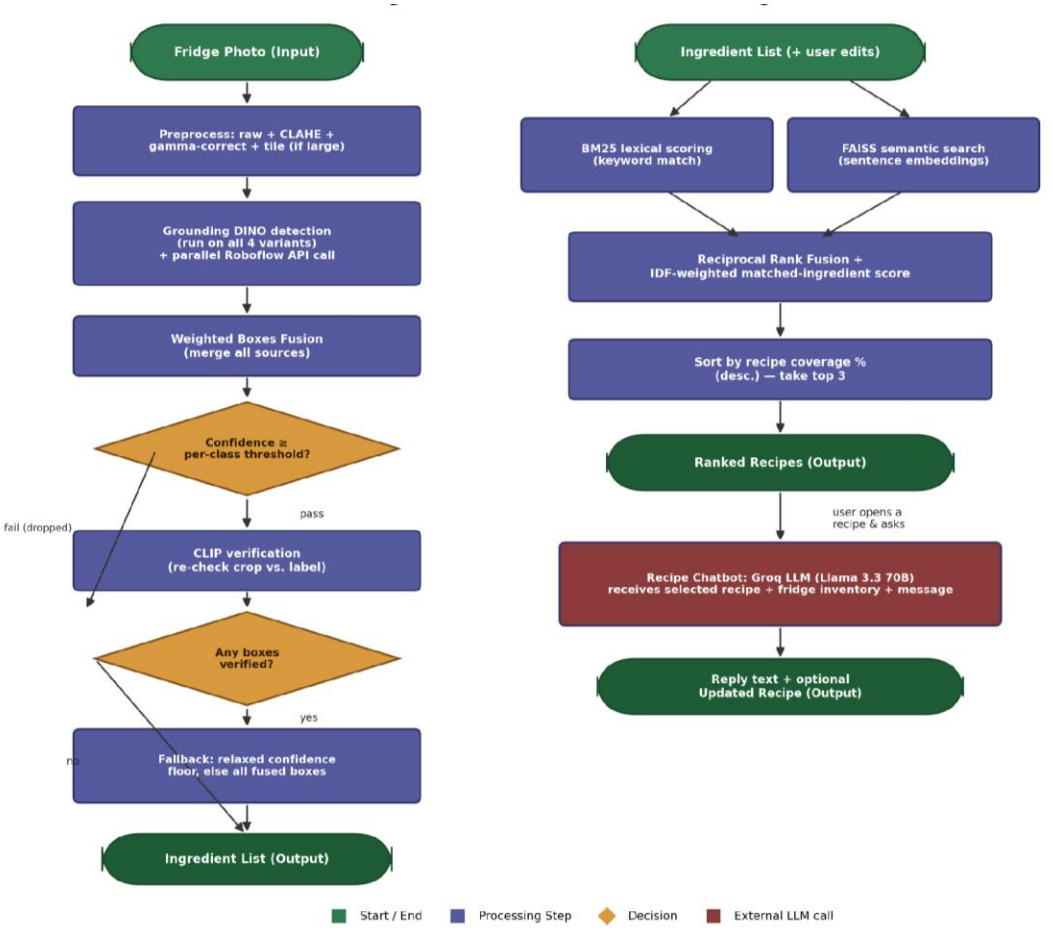

Figure 3:AI workflow overview: detection pipeline (left), recipe retrieval pipeline (right), and the chatbot step that consumes a selected recipe.

As shown in Figure 2.3, the ingredient detection workflow consists of multiple processing stages that improve the reliability of the detected results through verification and fallback mechanisms. Similarly, the recipe retrieval workflow combines lexical and semantic retrieval techniques to identify the most relevant recipes before ranking them according to their ingredient coverage. This integrated workflow enables the system to provide accurate recipe recommendations and support conversational recipe assistance.

CHAPTER 3

Artificial Intelligence and Bachend

3.1 Introduction

This chapter documents the core technical layers of the system: the Artificial Intelligence module, the backend service that exposes it, and the data layer that supports it. As established in the project overview, the AI module is the core of the system, and the frontend exists only to demonstrate it; this chapter is accordingly written in the greatest depth on the AI module. Every statement below is derived directly from the implementation found in App/AI/fridgo_backend.ipynb (the backend notebook), App/js/script.js (the frontend integration layer), and the accompanying HTML pages. Where a conventional documentation heading does not correspond to anything present in the code, this is stated explicitly rather than assumed.

3.2 Artificial Intelligence Module

3.2.1 Dataset

Two distinct datasets are involved in the system, serving two different sub-components of the AI module

- **Recipe dataset** — "recipe-dataset-over-2m" (Kaggle identifier: wilmerarltstrmberg/recipe-dataset-over-2m), downloaded programmatically at server startup via kagglehub.dataset_download(). The file recipes_data.csv is loaded into a pandas DataFrame. The columns actually used by the code are title, ingredients, directions, and NER. The system restricts itself to the first 15,000 rows that pass a cleaning filter (dropna, then head(15000)), and further removes any row whose NER-extracted ingredient list is empty.
- Fridge-item image data — used to train the object-detection model hosted on Roboflow under the identifier "fridge-detection-ojugs/6". This model is called only as a remote HTTP API in the code; no training images, annotation files, class-balance statistics, or training scripts for this model exist in the repository.

3.2.2 Data Preprocessing

For the recipe dataset (verified in the notebook's cell 1):

Rows with a missing title, ingredients, directions, or NER value are dropped using pandas dropna().

Only the first 15,000 valid rows are retained (head(15000)) — an explicit, hardcoded cap chosen by the developers, not the full dataset of over two million recipes.

Stringified list columns (ingredients, NER, directions) are parsed back into real Python lists by a safe_parse() helper built on ast.literal_eval, with a silent fallback to an empty list if parsing fails.

Rows whose parsed NER list is empty after this step are removed.

A composite search_text field is built by concatenating the recipe title and the raw ingredients string; this field is later used as the input to the embedding model.

Direction steps are cleaned and capped to at most 8 steps per recipe by get_directions_list(), which appends a "…and N more step(s)" marker when a recipe has more steps than the cap.

For the vision pipeline, "preprocessing" takes the form of generating three additional image variants per uploaded photo, rather than a training-time preprocessing step:

- The raw RGB image, used as-is.
- A CLAHE-enhanced version (apply_clahe): contrast-limited adaptive histogram equalization applied to the L-channel after converting to LAB color space.
- A gamma-corrected version (apply_gamma, γ = 1.6): brightens dark regions of the image via a precomputed lookup table.
- For images where max(height, width) exceeds 900 pixels, additional overlapping tiles are cropped using a sliding-window scheme (640px tiles, 20% overlap, capped at 4 tiles) and processed independently.

Each of these variants is passed independently through Grounding DINO before being recombined during fusion (see section 3.2.10).

3.2.3 Feature Engineering

The following feature-engineering steps are verified in the code:

- Text embeddings: the search_text field (title + ingredients) is encoded using a SentenceTransformer (all-MiniLM-L6-v2, max_seq_length = 256) into dense vectors, indexed with a FAISS IndexFlatL2 for semantic recipe search.
- Lexical tokens: each recipe's NER ingredient list is lower-cased and tokenized for indexing with BM25Okapi.
- Ingredient rarity (IDF): a custom inverse-document-frequency table is computed over the corpus of NER ingredient lists (ingredient_idf), so that rarer ingredients (e.g., salmon) contribute more weight to a recipe match than common ones (e.g., eggs).
- CLIP text prompts: for each of the target words, a natural-language template is constructed (for example, "a close-up photo of an apple in a fridge"), embedded once at startup through CLIP's text encoder, and reused for every verification call during inference — this is the only prompt-engineering step on the vision side.

3.2.4 Model Architecture

Model	Role in the system	Trained by this project?
Grounding DINO (IDEA-Research/grounding-dino-base)	Open-vocabulary object detector — locates and labels the target ingredient words in the uploaded image.	No — pretrained, used frozen (zero-shot).
CLIP (openai/clip-vit-base-patch32)	Verification layer — re-scores each detected crop against the text prompts to reject false positives.	No — pretrained, used frozen.
Custom Roboflow model (fridge-detection-ojugs/6)	A second, independently trained detector, called through Roboflow's hosted inference API.	Trained externally on Roboflow's platform. Architecture, backbone, and training configuration are not present in this repository.

Model	Role in the system	Trained by this project?
SentenceTransformer (all-MiniLM-L6-v2)	Embeds recipe text for semantic (FAISS) search.	No — pretrained, used frozen.
BM25Okapi	Statistical, non-neural lexical ranking algorithm.	Not applicable — classical information-retrieval algorithm, not a trained model.
Groq-hosted LLM (llama-3.3-70b-versatile)	Powers the conversational recipe assistant (/api/chat).	No — accessed exclusively through Groq's hosted completions API.

Because every model listed above is either a frozen pretrained network or a remote hosted API, the AI contribution of this project lies in the orchestration layer built around these models: the multi-pass image augmentation and tiling strategy, the Weighted Boxes Fusion ensembling step, the CLIP-based verification and fallback logic, and the hybrid BM25 + FAISS + IDF recipe-ranking formula — not the underlying model weights themselves. This distinction is stated here explicitly so it can be represented accurately and honestly in the written thesis.

3.2.5 Metrics

The only quantitative signals that the system computes and exposes at runtime are the following:

- Per-detection confidence score (conf) — the raw score assigned by Grounding DINO or Roboflow, retained after Weighted Boxes Fusion.
- CLIP similarity score (clip_score) — the cosine similarity between a detected crop's image embedding and its claimed label's text embedding.
- Recipe match_percentage — the percentage of a recipe's own ingredient list that the user's detected/edited ingredients cover, computed as recipe_coverage × 100 and rounded to the nearest integer.
- Weighted match score (weighted_match) and Reciprocal Rank Fusion score (rrf) — internal ranking signals used to order candidate recipes; these are not exposed to the frontend directly.

These are all per-request, runtime scores produced while serving a single user's photo or ingredient list. They are not aggregate model-quality metrics computed over a held-out test set, since no such evaluation exists in the code (section 3.2.7).

3.2.6 Hyperparameters

The table below lists every configurable numeric parameter found in the source code, together with its verified value and where it is used.

Parameter	Value	Used in
CLASS_THRESHOLDS (per class)	e.g. tomato 0.15, milk 0.20, salmon 0.18, cheese 0.18 (full 14-class dict in code)	Per-class confidence filter applied after box fusion
FALLBACK_MIN_CONF	0.12	Relaxed confidence threshold used only if CLIP verification rejects everything
CLIP_MARGIN	0.08	Similarity margin tolerance in clip_verify()

Parameter	Value	Used in
WBF iou_thr	0.5	Weighted Boxes Fusion IoU threshold
WBF skip_box_thr	0.05	Minimum score for a box to be kept during fusion
WBF source weights	[1.0, 0.8, 0.7, 0.9, 1.5]	Weights for full / CLAHE / gamma / tiles / Roboflow sources respectively
Tile size / overlap / max tiles	640 px / 0.2 / 4	sliding_windows() tiling parameters
Gamma correction γ	1.6	apply_gamma()
CLAHE clip limit / grid size	2.0 / (8, 8)	apply_clahe()
embed_model.max_seq_length	256	SentenceTransformer text encoding
FAISS search k	min(len(df), 500)	Candidate pool size for semantic recipe search
Recipe Top-K	3	Number of recipes returned per request
Roboflow call params	confidence 0.01, overlap 0.3, max_detections 50	Parameters sent to the Roboflow inference API
Groq temperature	0.4	call_groq_chat() LLM request
Groq max_tokens	1500	call_groq_chat() LLM request
Recipe dataset row cap	15,000	df.head(15000) at startup
Max direction steps shown	8	get_directions_list()
Max upload size	15 MB	/api/detect file-size validation

3.2.7 Prediction Pipeline

The system exposes two independent prediction pipelines: an ingredient-detection (vision) pipeline and a recipe-ranking pipeline. Both are described below and illustrated in Figures 3.1 and 3.2.

Vision detection pipeline (triggered by POST /api/detect): the uploaded photo is decoded and converted to RGB. Four image variants (full, CLAHE-enhanced, gamma-corrected, and — for large images — up to four overlapping tiles) are each passed through Grounding DINO independently, using the fixed 14-word text prompt. In parallel, the same image is resized and sent to the Roboflow serverless inference endpoint, whose raw predictions are cleaned with class-aware non-maximum suppression. All five detection sources are then merged with Weighted Boxes Fusion, using the source weights listed in section 3.2.9. The fused boxes are filtered by a per-class confidence threshold, and each surviving box is passed through the CLIP verification layer, which crops the region and compares it against the 14 CLIP text prompts; a box is accepted if CLIP's own top choice agrees with the claimed label, or if the similarity gap is within CLIP_MARGIN. If this step rejects everything, two fallback stages progressively relax the filtering so that the pipeline still returns a best-effort result rather than nothing. The final, de-duplicated set of ingredient labels is returned to the frontend together with an annotated copy of the image.

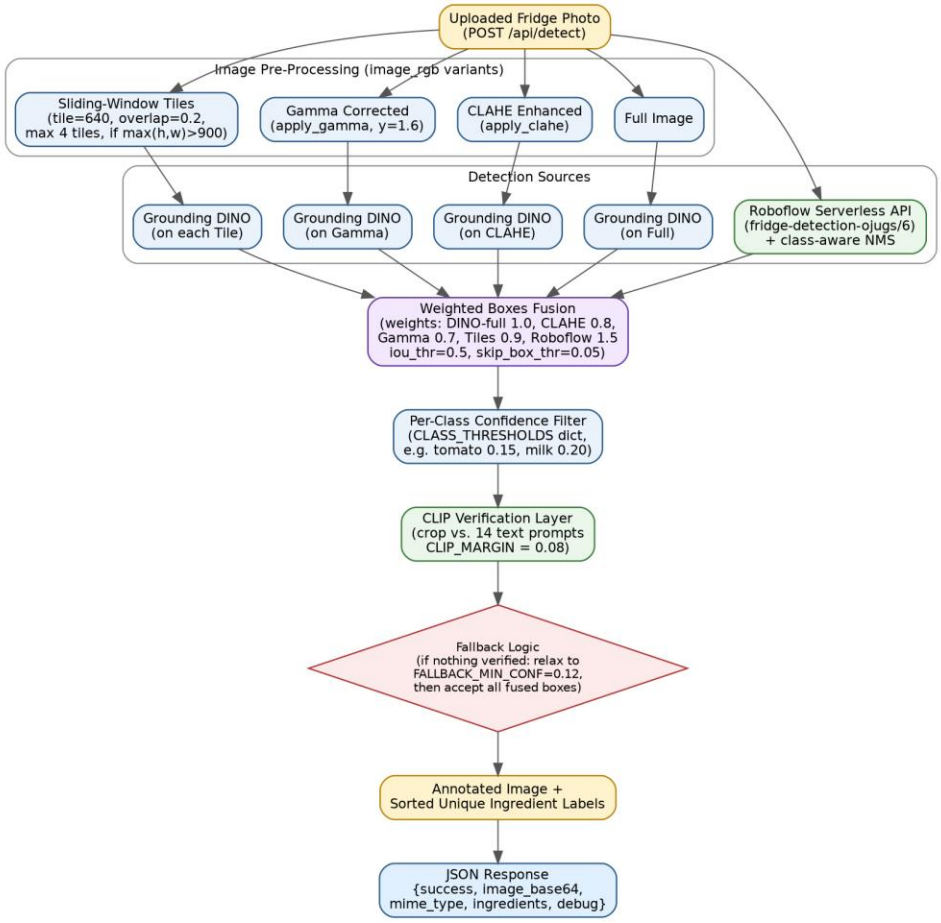

Figure4 :Ingredient detection (vision) prediction pipeline, generated from detect_ingredients_logic() and its helper functions in the backend notebook.

Recipe ranking pipeline (triggered by POST /api/recipes): the frontend sends the current, user-edited ingredient list. The backend runs this list through two independent retrieval methods over the same in-memory recipe index built at startup — a BM25 lexical search over each recipe's NER ingredient tokens, and a FAISS semantic search over MiniLM embeddings of each recipe's title and ingredients text. Candidates returned by either method are merged, and each is scored using Reciprocal Rank Fusion combined with the IDF-weighted rarity of its matched ingredients. The final sort key is the recipe's own ingredient coverage — the percentage of that recipe's ingredients the user actually has — with the weighted match, raw match count, and RRF score used only as tie-breakers. The top three recipes are returned to the frontend as a structured JSON array (title, ingredients, directions, match_percentage, matched_ingredients), with a markdown rendering also included for backward compatibility.

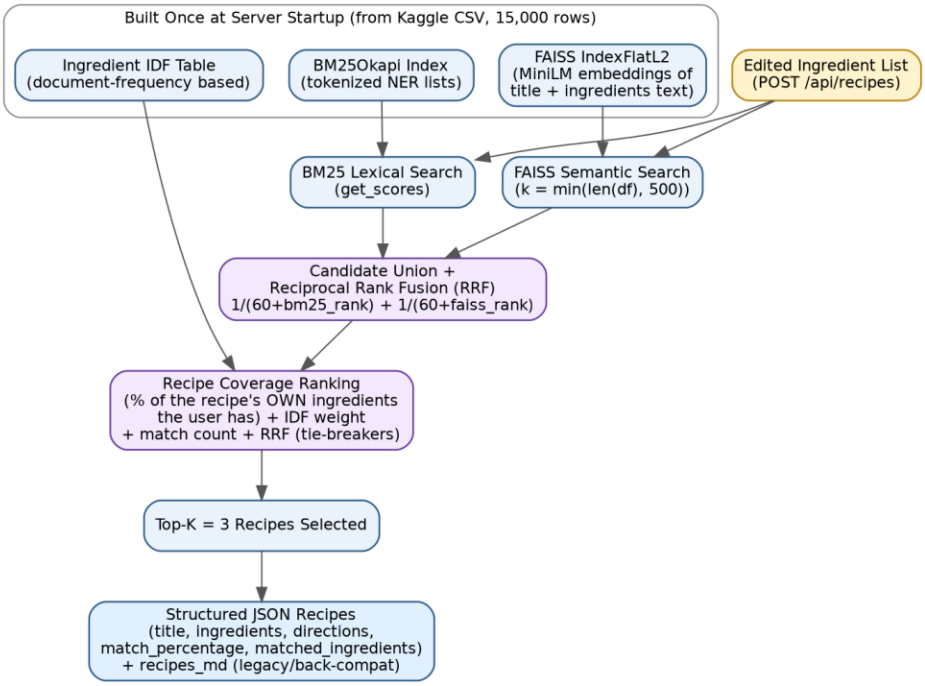

Figure5 : Recipe retrieval and ranking pipeline, generated from search_recipes_logic() as patched in the backend notebook's final cell.

3.2.8 Model Loading

All locally-run models are loaded once, synchronously, at process startup — inside the same notebook cell that later defines and starts the FastAPI application — before the server begins accepting requests. Verified loading steps:

- Grounding DINO: AutoProcessor and GroundingDinoForObjectDetection are loaded via from_pretrained("IDEA-Research/grounding-dino-base") and moved to the available device (cuda if a GPU is present, otherwise cpu).
- CLIP: CLIPModel and CLIPProcessor are loaded via from_pretrained("openai/clip-vit-base-patch32") and moved to the same device; the text embeddings for the 14 fixed prompts are computed once and cached in memory (CLIP_TEXT_FEATURES) for reuse on every request.
- SentenceTransformer: all-MiniLM-L6-v2 is loaded onto the same device and configured with max_seq_length = 256.
- Recipe index: the Kaggle CSV is downloaded and the FAISS and BM25 indexes are built once, in memory, immediately after the dataset is cleaned.

The Roboflow model and the Groq LLM are not "loaded" into the process at all — they are invoked remotely, per request, over HTTPS, and therefore have no local loading step or memory footprint on the Colab machine. There is no persistent model cache maintained by the project itself beyond what Hugging Face's and Kaggle's own client libraries cache internally; because the backend runs inside a Google Colab session, all in-memory structures (recipe index, cached embeddings, session dictionary) must be rebuilt from scratch every time the Colab runtime restarts.

3.2.8 Libraries

Category	Libraries (verified from imports / pip install cell)
Web framework / serving	fastapi, uvicorn, pyngrok, nest_asyncio, python-multipart
Deep learning runtime	torch, transformers (AutoProcessor, GroundingDinoForObjectDetection, CLIPModel, CLIPProcessor)
Computer vision	opencv-python-headless (cv2), supervision, ensemble-boxes (weighted_boxes_fusion)
Recipe search / NLP	sentence-transformers, faiss-cpu, rank_bm25
Data handling	pandas, numpy
External data / model sources	kagglehub (recipe dataset), Hugging Face Hub (model weights), requests (Roboflow + Groq HTTP calls)

3.2.9 Limitations

The following limitations are directly observable from the implementation:

- Dependency on three external network services at runtime — Roboflow, Groq, and the Hugging Face/Kaggle downloads performed at startup — any outage of these breaks the corresponding feature.
- Session memory (SESSION_LAST_RECIPE) is a plain in-memory Python dictionary; all chat/recipe context is lost whenever the backend process restarts.
- The recipe corpus used for ranking is capped at the first 15,000 of the source dataset's more than two million rows — a simple positional cut, not a curated or stratified sample.
- The backend is coupled to the Google Colab execution model (a threaded Uvicorn server plus an ngrok tunnel running inside a notebook process), which is not a conventional production deployment pattern.

3.3 Backend

3.3.1 Framework

The backend is built with FastAPI (Python) and served by Uvicorn. It runs inside a background thread of a single Google Colab process and is exposed to the public internet through an ngrok TLS tunnel. CORS middleware is configured permissively, accepting all origins, methods, and headers (allow_origins=["*"]). The entire backend — model loading, dataset preparation, route definitions, and server startup — lives in one executable notebook rather than a modular multi-file application.

3.3.2 APIs

Five HTTP endpoints are defined and verified in the code:

Method & Path	Request body	Response (key fields)	Purpose
GET /health	—	status, device, recipes_loaded	Liveness/status check
GET /api/config	—	valid_ingredients (the 14-word list)	Exposes the fixed detectable vocabulary

Method & Path	Request body	Response (key fields)	Purpose
POST /api/detect	multipart file (image)	success, image_base64, mime_type, ingredients[], debug	Runs the vision pipeline on an uploaded photo
POST /api/recipes	{ ingredients: string[] }	success, recipes[] (structured), recipes_md	Runs the hybrid recipe ranking pipeline
POST /api/chat	{ message, recipe, session_id, available_ingredients }	success, reply, updated_recipe	Recipe Q&A and recipe-modification chatbot

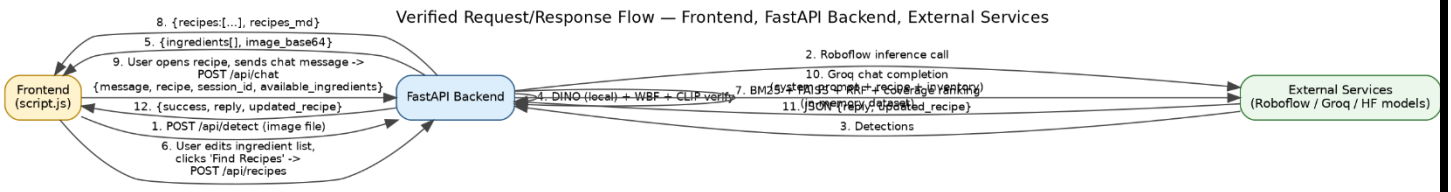

Figure6 : Verified request/response interaction flow across the frontend, the FastAPI backend, and the external services it calls for the three main user actions (detect, get recipes, chat)

The codebase does not use a formal Controller/Service/Repository layered architecture. All request handling is implemented directly as FastAPI route functions — health(), config(), detect_api(), recipes_api(), and chat_api() — defined inside the same script as everything else. There are no separate controller classes, modules, or routers; each route function acts as its own controller and directly invokes the corresponding processing logic.

3.3.4 Services

Although not organized into formal service classes, the following functions fulfil an equivalent business-logic role and are called directly by the route handlers:

- detect_ingredients_logic() — the vision service, orchestrating detection, fusion, and verification.
- search_recipes_logic() — the recipe ranking service (redefined in the notebook's final patch cell to rank by recipe-ingredient coverage).
- call_groq_chat() — the chat/LLM service (redefined in an earlier patch cell to also accept the detected fridge inventory as context).
- find_recipe_by_title() — a helper used by the chat service to ground the LLM in a real recipe from the dataset when the frontend does not supply one directly.

No dependency-injection framework, service registry, or repository abstraction is used; all functions are called directly as ordinary Python function calls within the same global module/notebook scope.

3.3.5 AI Integration

From the backend's perspective, AI integration takes three verified forms:

- In-process inference: Grounding DINO, CLIP, and the SentenceTransformer are loaded into the same Python process and invoked synchronously inside the request-handling code.
- Outbound HTTP calls to third-party AI APIs: the Roboflow inference endpoint (30-second timeout) and the Groq chat-completions endpoint (60-second timeout), both called using the requests library.
- No message queue, background task runner, or asynchronous job system is used anywhere — every AI call blocks the handling of its HTTP request until the call returns.

3.3.6 Error Handling

The following error-handling behavior is verified in the code:

- /api/detect rejects unsupported file content types with HTTP 400, rejects files larger than 15 MB with HTTP 413, rejects undecodable images with HTTP 400, and wraps the detection call in a try/except block that raises HTTP 500 with the underlying exception message on failure.
- /api/recipes rejects an empty ingredients list with HTTP 400, and returns success: false with an empty recipe list (rather than an error) when no recipes match.
- /api/chat rejects an empty message with HTTP 400; internally, call_groq_chat() catches all exceptions raised during the Groq request or its JSON parsing and returns a graceful fallback reply instead of propagating an error to the frontend.
- Startup-level errors — missing API keys, a failed Kaggle download, or an empty dataset after cleaning — raise a RuntimeError that halts the notebook cell, with diagnostic text printed to the Colab console.
- All error handling relies on FastAPI's built-in HTTPException and standard Python exception handling; there is no centralized/global exception-handling middleware, structured logging framework, or external error-tracking service. Diagnostics rely entirely on print() statements written to the Colab console output.

CHAPTER 4

Frontend Development

4.1 Overview

The frontend of Fridgo is a lightweight, static client built with plain HTML, CSS, and vanilla JavaScript (no frontend framework, build tool, or package manager was identified in the project). Its sole purpose is to provide a usable interface through which a user can interact with the AI backend — it contains no business logic of its own and performs no AI processing locally. All intelligence (ingredient detection, recipe generation, and conversational modification) is delegated entirely to backend API endpoints.

4.2 Pages

The frontend consists of three static HTML pages located in `App/pages/`:

Page	File	Purpose
Landing Page	`landing_page.html`	Marketing/entry page introducing the product, with a hero section, a static "How It Works" explanation (3 steps), and call-to-action buttons that redirect to the dashboard.
Main App Dashboard	`main_app_dashboard.html`	The functional core of the UI. Contains the image upload zone, detected inventory panel, recipe results grid, and an AI chat panel.
About Us	`about_us.html`	Static team/supervisor information page with no functional or AI-related behavior.

4.3 Components

The UI is composed of reusable structural blocks (styled via a shared `style.css`) rather than componentized in the framework sense (i.e., there is no component library or templating engine — each block is authored directly in HTML per page). The main functional components on the dashboard page are:

- **Upload Zone** (`#uploadZone`, `#fileInput`, `#browseBtn`): allows the user to select or drop a fridge photo; includes an image preview and a processing overlay with a spinner shown during detection.

- **Detected Inventory Panel** (`#inventoryTags`, `#inventoryCount`): displays ingredient tags returned by the AI, each removable via a close icon; also includes a manual "add item" input for correcting or supplementing AI results.
- **Recipe Suggestions Grid** (`#recipeGrid`): dynamically populated with recipe cards after the user requests recipes.
- **AI Chat Panel** (`.ai-body`, `.ai-input`, `.ai-send`): a chat-style interface allowing the user to ask the AI to modify a selected recipe (e.g., ingredient substitutions).
- **Shared Header/Footer**: present on all three pages, providing navigation links and branding.

4.4 Navigation

Navigation is implemented using standard anchor (`<a href>`) links between the three static HTML files — there is no client-side router or single-page-application behavior. The header on every page links to `landing_page.html`, `about_us.html`, and `main_app_dashboard.html`. In addition, `script.js` attaches a click handler to all elements with the class `.get-start` (used for "Get Started" buttons on the landing page) that programmatically redirects the user to `main_app_dashboard.html`.

4.5 API Integration

All communication with the AI backend is handled in `App/js/script.js` using the Fetch API. The base API URL is resolved from a global variable, `localStorage`, or a default fallback (an ngrok tunnel address), allowing the backend endpoint to be reconfigured without changing the code. Three REST endpoints are consumed:

Endpoint	Method	Triggered by	Payload	Purpose
`/api/detect`	POST	Selecting/uploading an image	`multipart/form-data` containing the image file	Sends the fridge photo to the backend for AI-based ingredient detection
`/api/recipes`	POST	Clicking "Find Recipes"	JSON `{ ingredients: [...] }`	Requests recipe suggestions based on the current ingredient list

`/api/chat`	POST	Sending a message in the AI chat panel	JSON `{ message, recipe, session_id, available_ingredients }`	Sends a conversational request (e.g., recipe modification) to the AI

Each request includes the `ngrok-skip-browser-warning` header to bypass ngrok's interstitial warning page, confirming the backend is exposed via an ngrok tunnel during development/testing. All three calls are wrapped in `try/catch` blocks with `finally` clauses that reset UI state (e.g., re-enabling buttons, hiding the processing overlay) regardless of success or failure, and user-facing error handling is provided for failed detection, failed recipe retrieval, and unreachable backend connections.

4.6 Displaying AI Results

- **Ingredient Detection Results:** Upon a successful response from `/api/detect`, the returned `ingredients` array is normalized (trimmed, lowercased) and rendered as removable tags in the inventory panel; the returned `image_base64` (if present) replaces the local image preview with the backend-processed image.
- **Recipe Results:** Upon a successful response from `/api/recipes`, the `renderRecipes()` function generates a recipe card for each returned recipe, displaying its title, a computed match percentage, matched ingredients, a full ingredient list, and step-by-step directions. The function supports both a structured array response and a markdown-style string response (recipes separated by `---`), indicating the frontend was built to tolerate variation in backend output format.
- **Chat Responses:** Messages sent to `/api/chat` are appended to the chat panel as user bubbles; while awaiting a response, a temporary "loading" message is shown and then replaced with the AI's `reply` field once received, or a fallback message if the backend is unreachable.

CHAPTER 5

Project Deployment and Execution

5.1 Software Requirements

- **AI Backend:** Python 3 environment with GPU access, developed and executed using Google Colab with GPU acceleration (T4 recommended).
- **Frontend:** Any modern web browser. No build tools, Node.js, or web framework are required, as the frontend is static HTML/CSS/JS.

5.2 Dependencies

The project dependencies are installed directly within the notebook using pip

The AI backend depends on the following Python libraries:

- **Web Framework:** FastAPI, Uvicorn
- **Deployment:** Pyngrok, Nest Asyncio
- **API Support:** Python Multipart
- **Computer Vision:** OpenCV, Supervision, Ensemble Boxes
- **Machine Learning:** PyTorch, Transformers
- **Recipe Retrieval:** Sentence Transformers, FAISS, Rank-BM25
- **Data Processing:** Pandas, NumPy
- **Dataset Access:** KaggleHub
- **Utilities:** Requests

The frontend has no additional software dependencies beyond Google Fonts and Material Symbols Icons, which are accessed through their respective CDNs.

5.3 Environment Variables / Secrets

Three secrets are required by the backend and are retrieved via Google Colab's Secrets Manager (`google.colab.userdata`), an OS environment variable, or an interactive prompt (`getpass`) as a fallback:

Secret	Purpose	Required?
`ROBOFLOW_API_KEY`	Authenticates calls to the hosted Roboflow object-detection model	Yes — backend raises an error and stops if missing
`NGROK_AUTH_TOKEN`	Authenticates the ngrok tunnel used to expose the backend publicly	Yes — backend raises an error and stops if missing
`GROQ_API_KEY`	Authenticates calls to the Groq LLM API used by `/api/chat`	Optional — if missing, `/api/chat` returns a "not configured" message instead of failing

The frontend communicates with the backend through a configurable API endpoint. During deployment, this endpoint is set to the public URL generated by the ngrok tunnel, enabling communication between the frontend and the AI backend.

5.4 Installation Steps

1. Open the AI backend notebook in Google Colab.
2. Select a GPU runtime (T4 or higher is recommended).
3. Run the dependency installation cell.
4. Configure the required API keys.
5. The frontend requires no installation, as it consists of static HTML, CSS, and JavaScript files.

5.5 Running the Backend

After the required dependencies are installed and the API keys are configured, the backend can be started by running the main notebook. During initialization, the system loads the AI models, prepares the recipe retrieval components, starts the FastAPI server, and exposes the backend through an ngrok tunnel. A public HTTPS URL is generated and used by the frontend to communicate with the backend throughout the execution of the application.

5.6 Running the Frontend

The frontend consists of static HTML pages and can be opened directly in any modern web browser or served using a simple static web server. Before using the application, the frontend should be configured to communicate with the backend using the public URL generated during the backend deployment. Once configured, users can upload refrigerator images, browse recipe recommendations, and interact with the conversational assistant through the web interface.

5.7 Running the AI Model

The AI models operate as integrated components of the backend services and are executed automatically when requests are received from the frontend. Depending on the requested operation, the backend invokes the appropriate AI workflow for ingredient detection, recipe recommendation, or conversational assistance. The models are therefore not executed independently but as part of the complete application workflow.

5.8 Project Execution Workflow

1. Start the AI backend in Google Colab and obtain the public backend URL generated through the ngrok tunnel.
2. Configure the frontend to communicate with the backend using the generated URL.
3. Open the main application page in a web browser.
4. Upload a refrigerator image for ingredient detection.
5. Review the detected ingredients and edit the list if necessary by adding or removing items.
6. Search for recipes based on the final ingredient list.
7. Browse the recommended recipes and select one for further interaction.
8. Use the conversational assistant to ask questions about the selected recipe or request modifications based on personal preferences or dietary requirements.

CHAPTER 6

Challenges

6.1 Dependency Conflict (Keras 3 vs. Transformers)

The most concrete, evidenced challenge is a library incompatibility: the notebook's saved output shows that importing `sentence_transformers` (which pulls in `transformers.integrations`) failed because the installed Keras version (3.x) is incompatible with `transformers`' TensorFlow integration code, which expects Keras 2 (`tf_keras`). This halted execution of the entire backend cell.

6.2 Exposing a Local/Colab Backend to a Static Frontend

An integration challenge inherent to the architecture: since the AI backend runs inside a Google Colab notebook rather than a persistent server, the frontend needs a way to reach it. **How it was solved:** the backend uses `pyngrok` to open a public HTTPS tunnel to its local `uvicorn` server, and the frontend resolves the backend's address dynamically at runtime from `localStorage` or a global variable rather than a hardcoded value, allowing the ngrok URL to be updated each time a new tunnel is created.

6.3 Model Selection

One of the main challenges was selecting the most suitable deep learning model for ingredient detection. Several CNN architectures were considered, including lightweight and high-accuracy models, each with different trade-offs in terms of accuracy, training time, computational cost, and model size. The selected model needed to achieve reliable detection while remaining efficient enough for practical use.

CHAPTER 7

Future Work

Chapter 7: Future Work

- **Supporting additional datasets:** loading the full ~2-million-row recipe dataset instead of the current 15,000-row subset, or incorporating additional/regional recipe sources to broaden recipe diversity.
- **Better deployment:** replacing the Colab-notebook-plus-ngrok-tunnel setup with a persistent, dedicated server deployment (e.g., a containerized cloud service), removing the dependency on keeping a notebook tab open.
- **More advanced user interface:** adding an in-app settings panel for configuring the backend URL (removing the current need to use `localStorage` via developer tools), and persisting chat/session history beyond a single in-memory server session.

CHAPTER 8

Conclusion

The objective of this graduation project, Fridgo, was to build a system in which a user could photograph the contents of their refrigerator and receive AI-generated ingredient detection, recipe suggestions, and conversational recipe assistance, with the frontend serving purely as a demonstration layer for the underlying AI capabilities.

On the AI side, a multi-stage detection pipeline was designed and implemented, combining an open-vocabulary object detector (Grounding DINO), image-enhancement passes (CLAHE and gamma correction), image tiling for large inputs, a supplementary hosted Roboflow detection model, Weighted Boxes Fusion to merge these sources, and a CLIP-based verification step to filter false positives. A separate recipe-retrieval engine was also implemented, combining BM25 lexical search and FAISS-based semantic search over a Kaggle recipe dataset, fused via reciprocal rank fusion and IDF-weighted ingredient matching, to return ranked, structured recipe suggestions. A conversational assistant, backed by a hosted Groq LLM, was implemented to answer recipe-related questions and apply user-requested recipe modifications, grounded in both the currently open recipe and the user's detected inventory. All of this was exposed through a FastAPI backend with three functional endpoints (`/api/detect`, `/api/recipes`, `/api/chat`), consumed by a static HTML/CSS/JavaScript frontend that uploads images, displays detected ingredients as editable tags, renders recipe cards, and provides a chat interface — with real, working Fetch-based integration code for all three endpoints.

The main contribution of the AI module lies in this layered detection-and-retrieval design: rather than relying on a single object detector, the system was engineered to cross-verify detections across multiple models and image transformations before accepting them, and rather than relying purely on keyword or purely on semantic recipe search, it combines both retrieval strategies with an ingredient-rarity weighting scheme. The iterative "PATCH" cells found in the notebook also demonstrate a genuine, evidence-based development process, in which real usability problems (unparseable recipe output, a misleading match-percentage metric, a context-blind chatbot) were identified and corrected.